

Question_8:

Code:

```
1 // Given a tree with n nodes, find the maximum number of edges
2 // that can be selected so that no two selected edges share a node.
3 #include <iostream>
4 using namespace std;
5
6 #define MAXN 200005
7
8 struct Edge
9 {
10     int to;
11     Edge* next;
12 };
13 Edge* head[MAXN];
14 int dp[MAXN][2];
15
16 void addEdge(int u, int v)
17 {
18     Edge* e1 = new Edge;
19     e1->to = v;
20     e1->next = head[u];
21     head[u] = e1;
22     Edge* e2 = new Edge;
23     e2->to = u;
24     e2->next = head[v];
25     head[v] = e2;
26 }
27 void link(int i, int j)
28 {
29     addEdge(i, j);
30 }
31 void dfs(int u, int p)
32 {
33     dp[u][0] = 0;
34     dp[u][1] = 0;
35
36     int sum_max = 0;
37     for (Edge* e = head[u]; e != NULL; e = e->next)
38     {
```

```
39         int v = e->to;
40         if (v != p)
41         {
42             dfs(v, u);
43             int best_child = (dp[v][0] > dp[v][1]) ? dp[v][0] : dp[v][1];
44             sum_max += best_child;
45         }
46     }
47     dp[u][0] = sum_max;
48     for (Edge* e = head[u]; e != NULL; e = e->next)
49     {
50         int v = e->to;
51         if (v != p)
52         {
53             int best_child = (dp[v][0] > dp[v][1]) ? dp[v][0] : dp[v][1];
54             int take_edge = 1 + dp[v][0] + (sum_max - best_child);
55
56             if (take_edge > dp[u][1])
57                 dp[u][1] = take_edge;
58         }
59     }
60 }
61 int main()
62 {
63     int n;
64     if (cin >> n)
65     {
66         for (int i = 1; i <= n; i++)
67             head[i] = NULL;
68         for (int i = 0; i < n - 1; i++)
69         {
70             int u, v;
71             cin >> u >> v;
72             link(u, v);
73         }
74         dfs(1, 0);
75         int ans = (dp[1][0] > dp[1][1]) ? dp[1][0] : dp[1][1];
76         cout << ans << "\n";
77     }
78     return 0;
79 }
```

Output:

Test Case_1:

```
5
1 2
1 3
3 4
3 5
2
PS C:\Users\Prabin Yadav\
```

Test Case_2:

```
5
1 1
1 2
2 3
3 4
6
PS C:\Users\Prabin Yadav\
```